HNDIT1032 Week 02 Study Guide: Data Representation in Computers

Oct 28, 2026 · @Zakee

How to use this guide

This guide covers everything in Week 02 (Data Representation in Computers) plus fully worked answers for Tutorial 02. Read sections 1 to 6 in order, try each tutorial question yourself, then check your working against section 7.

By the end of this week you should be able to:

1. Explain why computers store everything as bits (0 and 1).
2. Name and size the data units: bit, nibble, byte, word, KB, MB, GB.
3. Tell positional (weighted) number systems apart from non-positional ones.
4. Convert numbers between decimal, binary, octal and hexadecimal, in every direction.
5. Encode decimal digits in BCD and text in ASCII.
6. Compare the character coding schemes: BCD, ASCII, Extended ASCII, EBCDIC and Unicode.

Next week builds directly on this: Logic Gates, the building blocks of digital systems.

1. Digital data and data units

A computer is an electronic device, so it can only tell two states apart: electricity present (1, on, high voltage) or absent (0, off, low voltage). Each 0 or 1 is a **bit** (binary digit). Every number, letter, picture and sound is ultimately stored as a pattern of bits, for example 1011.

Unit	Size	Notes
Bit	1 binary digit (0 or 1)	Smallest unit of information
Nibble	4 bits	Half a byte; one hex digit
Byte	8 bits	Smallest addressable unit of storage; two nibbles (high-order and low-order)

Unit	Size	Notes
Word	16 bits (in this course)	In general, a word is a CPU-dependent group of bytes: 16, 32 or 64 bits are common
Double word	32 bits	Two words
Kilobyte (KB)	1024 bytes	2^10 bytes
Megabyte (MB)	1024 KB	2^20 bytes = 1,048,576 bytes
Gigabyte (GB)	1024 MB	2^30 bytes

Worked example. How many bits are in 2 KB? 2 × 1024 bytes × 8 bits = 16,384 bits.

Remember: n bits can represent 2^n different values. 4 bits → 16 values, 8 bits → 256 values.

Computers represent two kinds of data: **numbers** (1, 235, -10, 0) and **characters** (A, a, ?, @). Sections 2 to 5 cover numbers; section 6 covers characters.

2. Number systems

Number systems are either **positional (weighted)**, where a digit's value depends on where it sits, or **non-positional (non-weighted)**, where it does not.

Non-positional systems give each symbol a fixed value regardless of position. Examples: Roman numerals and the Egyptian number system. Roman symbols are I = 1, V = 5, X = 10, L = 50, C = 100, D = 500, M = 1000 (so IV = 4, IX = 9, XL = 40). They are awkward for arithmetic, which is why computers do not use them.

Positional systems give every digit position a **weight**, which is a power of the base (radix). The value of a number is the sum of digit × weight:

```
N = d_{n-1} b^{n-1} + \dots + d_2 b^2 + d_1 b^1 + d_0 b^0
```

For example, decimal 345 = 3 × 10^2 + 4 × 10^1 + 5 × 10^0. The rightmost digit is position 0 (least significant); the leftmost is the most significant.

System	Base	Digits used	Weight of each position	Written as
Binary	2	0, 1	1, 2, 4, 8, 16, 32, 64, 128 …	1011_2

System	Base	Digits used	Weight of each position	Written as
Octal	8	0 – 7	1, 8, 64, 512, 4096 …	234_8
Decimal	10	0 – 9	1, 10, 100, 1000 …	156_{10}
Hexadecimal	16	0 – 9, A – F	1, 16, 256, 4096, 65536 …	$9C_{16}$

Key facts:

- **Binary** is what the hardware uses; every binary number is a sum of powers of 2.
- **Octal**: one octal digit = exactly 3 bits (because 8 = 2^3).
- **Hexadecimal**: one hex digit = exactly 4 bits, i.e. one nibble (because 16 = 2^4). A = 10, B = 11, C = 12, D = 13, E = 14, F = 15. It is also called an alphanumeric number system because it mixes digits and letters.
- Octal and hex exist because they are short, human-friendly ways of writing long binary strings.
- Always write the base as a subscript (156_{10}, 234_8) or in brackets, (234)8, so the reader knows which system you mean.

3. Decimal → binary, octal, hexadecimal

Use **repeated division by the target base**. Write down each remainder, keep dividing the quotient until it reaches 0, then read the remainders **from bottom to top**.

1. Divide the decimal number by the base (2, 8 or 16).
2. Write the remainder beside it; the quotient goes on the next line.
3. Repeat until the quotient is 0.
4. Read remainders upward: the last remainder is the most significant digit.
5. For hex, write remainders 10 – 15 as A – F.

Example A: 156_{10} → binary

Division	Quotient	Remainder
156 ÷ 2	78	0
78 ÷ 2	39	0
39 ÷ 2	19	1

Division	Quotient	Remainder
19 ÷ 2	9	1
9 ÷ 2	4	1
4 ÷ 2	2	0
2 ÷ 2	1	0
1 ÷ 2	0	1

Reading bottom to top: **$156_{10} = 10011100_2$** (the slide shows 010011100_2; a leading 0 does not change the value).

Example B: 156_{10} → octal. 156 ÷ 8 = 19 r **4**; 19 ÷ 8 = 2 r **3**; 2 ÷ 8 = 0 r **2**. Read upward: **$156_{10} = 234_8$**.

Example C: 156_{10} → hexadecimal. 156 ÷ 16 = 9 r **12 (C)**; 9 ÷ 16 = 0 r **9**. Read upward: **$156_{10} = 9C_{16}$**.

Check your answer by converting back (section 4): 2 × 64 + 3 × 8 + 4 = 156. ✓

Alternative for binary: subtract powers of 2. Find the largest power of 2 that fits, put a 1 there, subtract, repeat. 156 = 128 + 16 + 8 + 4 → bits at 128, 16, 8, 4 → 10011100_2. This is often faster in exams.

4. Binary, octal, hexadecimal → decimal

Use **positional weights**: multiply each digit by base^position (position 0 on the right) and add everything up.

1. Number the positions from right to left, starting at 0.
2. Multiply each digit by base^position.
3. Add the products.

Example A: 11001_2 → decimal

Digit	1	1	0	0	1
Weight	2^4 = 16	2^3 = 8	2^2 = 4	2^1 = 2	2^0 = 1
Product	16	8	0	0	1

16 + 8 + 0 + 0 + 1 = **25_{10}**. Shortcut for binary: just add the weights wherever there is a 1.

Example B: 325_8 → decimal

3 × 8^2 + 2 × 8^1 + 5 × 8^0 = 3 × 64 + 2 × 8 + 5 × 1 = 192 + 16 + 5 = **213_{10}**

Example C: 2056_{16} → decimal

2 × 16^3 + 0 × 16^2 + 5 × 16^1 + 6 × 16^0 = 8192 + 0 + 80 + 6 = **8278_{10}**

Powers worth memorising:

Power	0	1	2	3	4	5	6	7	8	9	10
2^n	1	2	4	8	16	32	64	128	256	512	1024
8^n	1	8	64	512	4096						
16^n	1	16	256	4096	65536						

5. Shortcut conversions: binary ↔ octal ↔ hexadecimal

Because 8 = 2^3 and 16 = 2^4, you never need to go through decimal. Group bits in **3s for octal** and **4s for hex**, always starting from the **right**, and pad the leftmost group with leading zeros.

Octal	Binary		Hex	Binary	Hex	Binary
0	000		0	0000	8	1000
1	001		1	0001	9	1001
2	010		2	0010	A	1010
3	011		3	0011	B	1011
4	100		4	0100	C	1100
5	101		5	0101	D	1101
6	110		6	0110	E	1110
7	111		7	0111	F	1111

Octal → binary: replace each octal digit with its 3-bit group. $(53)_8$ → 101 011 → **101011_2**.

Hex → binary: replace each hex digit with its 4-bit group. $(F3)_{16}$ → 1111 0011 → **11110011_2**.

Binary → octal: group in 3s from the right. 1100011_2 → 001 100 011 → **143_8**.

Binary → hex: group in 4s from the right. 100111100_2 → 0001 0011 1100 → **$13C_{16}$**.

Octal → hex (via binary): 752_8 → 111 101 010 → regroup in 4s: 0001 1110 1010 → **$1EA_{16}$**.

Hex → octal (via binary): $E8A_{16}$ → 1110 1000 1010 → regroup in 3s: 111 010 001 010 → **7212_8**.

The golden rule: group from the **right**. Grouping from the left is the most common mistake in this topic.

6. Character representation (coding schemes)

Text is stored as bits too. A **coding scheme** is an agreed table that maps each character to a bit pattern.

Scheme	Bits	Characters	Key points
BCD (Binary Coded Decimal)	4 per decimal digit (6-bit BCD character code: 2^6 = 64 characters)	10 digits (or 64 characters)	Each decimal digit is coded separately
ASCII (American Standard Code for Information Interchange)	7	2^7 = 128	Standard from ANSI; A = 1000001 = 65
Extended ASCII	8	2^8 = 256	Introduced by IBM in 1981 for its first PC; A = 01000001; not enough for all languages
EBCDIC (Extended Binary Coded Decimal Interchange Code)	8	256	Developed by IBM; used mainly on IBM mainframes; rarely used today
Unicode	16 (original)	2^16 = 65,536	Universal encoding; supports all written languages (Sinhala, Tamil, etc.); most common on the web; code points written U+0061

BCD (8421 code)

Replace **each decimal digit separately** with its 4-bit binary value. Do not convert the whole number.

7093_{10} → 7 = 0111, 0 = 0000, 9 = 1001, 3 = 0011 → **0111 0000 1001 0011** (BCD)

Codes 1010 to 1111 are never used in BCD, because no decimal digit is above 9.

ASCII

Every character has a unique code. Learn three anchors and count from them:

Range	First code (decimal)	First code (hex)	Example
Space	32	20	—
Digits 0 – 9	48	30	'5' = 53
Uppercase A – Z	65	41	'C' = 67
Lowercase a – z	97	61	'c' = 99

Lowercase = uppercase + 32. To get binary, convert the decimal code (7 bits for ASCII, 8 bits padded with a leading 0 for Extended ASCII).

Worked example: "Hello, world"

Char	H	e	l	l	o	,	(space)	w	o	r	l	d
Decimal	72	101	108	108	111	44	32	119	111	114	108	100
Hex	48	65	6C	6C	6F	2C	20	77	6F	72	6C	64
Binary	01001000	01100101	01101100	01101100	01101111	00101100	00100000	01110111	01101111	01110010	01101100	01100100

Unicode

Unicode assigns every character an integer code point, written in hex with a U+ prefix. Lowercase 'a' is U+0061 (decimal 97, the same as ASCII). The first 128 Unicode code points are identical to ASCII. On Windows, open **Character Map** to browse codes. Modern Unicode has grown far beyond 16 bits (over a million possible code points, stored using encodings such as UTF-8 and UTF-16), but for this course remember "16 bits, about 65,000 characters".

7. Tutorial 02: worked solutions

Every answer below was verified by computer. Try each question first, then compare your working, not just the final answer.

Q1. Decimal → binary, octal, hexadecimal

Decimal	Binary (÷2 remainders, read up)	Octal (÷8)	Hex (÷16)
a) 24	24→12 r0, 6 r0, 3 r0, 1 r1, 0 r1 → **11000_2**	24÷8 = 3 r0; 3÷8 = 0 r3 → **30_8**	24÷16 = 1 r8; 1 → **18_{16}**
b) 47	47→23 r1, 11 r1, 5 r1, 2 r1, 1 r0, 0 r1 → **101111_2**	47÷8 = 5 r7; 5 → **57_8**	47÷16 = 2 r15 (F); 2 → **$2F_{16}$**
c) 675	675 = 512 + 128 + 32 + 2 + 1 → **1010100011_2**	675÷8 = 84 r3; 84÷8 = 10 r4; 10÷8 = 1 r2; 1 → **1243_8**	675÷16 = 42 r3; 42÷16 = 2 r10 (A); 2 → **$2A3_{16}$**
d) 89	89 = 64 + 16 + 8 + 1 → **1011001_2**	89÷8 = 11 r1; 11÷8 = 1 r3; 1 → **131_8**	89÷16 = 5 r9; 5 → **59_{16}**

Q2. Binary → decimal

Binary	Working (add the weights of the 1s)	Decimal
a) 110000111	256 + 128 + 4 + 2 + 1	**391**
b) 110011	32 + 16 + 2 + 1	**51**
c) 1001111	64 + 8 + 4 + 2 + 1	**79**
d) 11000001	128 + 64 + 1	**193**

Q3. Octal → decimal

Octal	Working	Decimal
a) 234	2 × 64 + 3 × 8 + 4 × 1 = 128 + 24 + 4	**156**
b) 36	3 × 8 + 6 × 1 = 24 + 6	**30**

Octal	Working	Decimal
c) 456	4 × 64 + 5 × 8 + 6 × 1 = 256 + 40 + 6	**302**
d) 217	2 × 64 + 1 × 8 + 7 × 1 = 128 + 8 + 7	**143**

Q4. Hexadecimal → decimal

Hex	Working	Decimal
a) E16	14 × 256 + 1 × 16 + 6 × 1 = 3584 + 16 + 6	**3606**
b) 389	3 × 256 + 8 × 16 + 9 × 1 = 768 + 128 + 9	**905**
c) 2AB	2 × 256 + 10 × 16 + 11 × 1 = 512 + 160 + 11	**683**
d) FF	15 × 16 + 15 × 1 = 240 + 15	**255**

Q5. Binary → octal (groups of 3 from the right)

Binary	Grouped	Octal
a) 1100011	001 100 011	**143_8**
b) 110011001100	110 011 001 100	**6314_8**
c) 100111100	100 111 100	**474_8**
d) 110000011	110 000 011	**603_8**
e) 110011011	110 011 011	**633_8**

Q6. Binary → hexadecimal (groups of 4 from the right)

Binary	Grouped	Hex
a) 11000011111	0110 0001 1111	**$61F_{16}$**
b) 1100110011	0011 0011 0011	**333_{16}**
c) 100111100	0001 0011 1100	**$13C_{16}$**
d) 1100000100	0011 0000 0100	**304_{16}**

Binary	Grouped	Hex
e) 11001101110	0110 0110 1110	**$66E_{16}$**

Q7. Octal → binary (each digit → 3 bits)

Octal	Expanded	Binary
a) 25	010 101	**10101_2**
b) 65	110 101	**110101_2**
c) 34	011 100	**11100_2**
d) 267	010 110 111	**10110111_2**

Q8. Hexadecimal → binary (each digit → 4 bits)

Hex	Expanded	Binary
a) A1	1010 0001	**10100001_2**
b) 2AB	0010 1010 1011	**1010101011_2**
c) 239	0010 0011 1001	**1000111001_2**
d) CCD	1100 1100 1101	**110011001101_2**

Part e) is blank on the tutorial sheet, so there is nothing to answer.

Q9. Decimal → BCD (and pure binary for comparison)

The question says "BCD into binary", but the values given are ordinary decimal numbers. The intended task is to **write each number in BCD** (each digit → 4 bits). The pure binary value is shown alongside so you can see the two are different.

Decimal	BCD (digit by digit)	Pure binary (whole number)
a) 278	**0010 0111 1000**	100010110_2
b) 405	**0100 0000 0101**	110010101_2

Decimal	BCD (digit by digit)	Pure binary (whole number)
c) 150	**0001 0101 0000**	10010110_2
d) 3284	**0011 0010 1000 0100**	110011010100_2

Q10. Words → ASCII (A = 65)

Count forward from A = 65: B = 66, C = 67, E = 69, L = 76, P = 80, T = 84.

Word	Decimal codes	7-bit binary	Hex
a) ABC	65 66 67	1000001 1000010 1000011	41 42 43
b) CAT	67 65 84	1000011 1000001 1010100	43 41 54
c) APPLE	65 80 80 76 69	1000001 1010000 1010000 1001100 1000101	41 50 50 4C 45

8. Common mistakes, slide corrections and exam tips

Common mistakes

- Reading division remainders top to bottom. Always read **bottom to top**.
- Grouping bits from the left. Always group from the **right** and pad the left with zeros.
- Writing 10 – 15 as two digits in hex (e.g. "2 15" instead of 2F).
- Using digits that do not exist in the base: 8 or 9 in octal, or 2 in binary.
- Confusing BCD with binary. 15_{10} in BCD is 0001 0101; in binary it is 1111.
- Forgetting the base subscript, so the examiner cannot tell 101_2 from 101_{10}.
- Mixing up uppercase and lowercase ASCII: 'A' = 65 but 'a' = 97.

Corrections to the Week 02 slides

Slide	What it says	Correct version
10 (Roman numerals)	C = 50, M = 100	L = 50, C = 100, D = 500, M = 1000
32 (ASCII)	American Standard Code for Information Exchange	American Standard Code for Information **Interchange**

Slide	What it says	Correct version
34 ("Hello, world")	Second 'o' shown as 01100111 / 67 / 103	That is 'g'. 'o' = 01101111 / 6F / 111
30 (BCD)	BCD uses 6 bits, 64 characters	That is the 6-bit BCD character code. Standard 8421 BCD for numbers uses 4 bits per decimal digit, as in slide 31
37 (Unicode)	16 bits, 65,000 characters	2^16 = 65,536. Modern Unicode exceeds 16 bits (over 1.1 million code points)
7 (Word)	Word = 16 bits	True by this course's convention; in general word size depends on the CPU (16, 32 or 64 bits), as slide 6 says

Exam tips

- Show every division step or every weight multiplication; method marks are given even if the final answer slips.
- Check every conversion by converting back the other way.
- For decimal → octal or hex of large numbers, go decimal → binary → group. It is often faster and less error-prone.
- Memorise the 4-bit table (0000 to 1111 → 0 to F). It unlocks every binary/octal/hex question.
- In a definition question, give the base, the digits allowed and one example.

9. Quick reference and self-test

Which method for which conversion

From → To	Method
Decimal → any base	Repeated division by the base; read remainders bottom to top
Any base → decimal	Multiply each digit by base^position; add

From → To	Method
Binary → octal / hex	Group in 3s / 4s from the right
Octal / hex → binary	Replace each digit with 3 / 4 bits
Octal ↔ hex	Go through binary and regroup
Decimal → BCD	Each decimal digit → 4 bits, separately
Character → ASCII	Look up or count from A = 65, a = 97, 0 = 48

Self-test (answers below; cover them first)

1. Convert 100_{10} to binary, octal and hexadecimal.
2. Convert 10110110_{2} to decimal, octal and hexadecimal.
3. Convert 777_{8} to decimal and hexadecimal.
4. Convert $3F_{16}$ to decimal and octal.
5. Write 2026_{10} in BCD.
6. Write "Hi" in ASCII (decimal and 7-bit binary).
7. How many different characters can an 8-bit code represent?
8. Convert $1A3_{16}$ to octal.

Answers

1. 1100100_{2}, 144_{8}, 64_{16}
2. 182_{10}, 266_{8}, $B6_{16}$
3. 511_{10}, $1FF_{16}$
4. 63_{10}, 77_{8}
5. 0010 0000 0010 0110
6. H = 72 = 1001000, i = 105 = 1101001
7. 2^8 = 256
8. 0001 1010 0011 → 000 110 100 011 → 643_{8}